STATE-SPACE SEARCH

Any-Path Search

Progressive Deepening Search

n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

Space (the final frontier)

In large search problems, **memory** is often the **limiting factor**.

BFS

• Imagine searching a tree with **branching factor 8** and **depth 10**. Assume a **node** requires just **8 bytes of storage**. Then, Breadth-First search might require up to:

$(2^3)^{10} * 2^3 = 2^{33}$ bytes $= 8{,}000$ Mbytes $= 8$ Gbytes

Recall Worst case space for Breadth-First Search 

Progressive Deepening Search

Depth-First Search (DFS) has **small space requirements** (linear in depth), but has major problems:

- DFS can run forever in search space with infinite length paths
- DFS does not guarantee finding the **shallowest** goal

Progressive Deepening Search

Best of Both Worlds

Depth-First Search (DFS) has small space requirements (linear in depth), but has major problems:

- DFS can run forever in search space with infinite length paths
- DFS does not guarantee finding the shallowest goal

Breadth-First Search (BFS) guarantees finding the **shallowest goal states**, even in the presence of infinite paths, but has one great problem:

- BFS requires a great deal of space (exponential in depth)

SEARCH

Space (the final frontier)

PDS

One strategy is to trade time for memory.
For example, we can emulate Breadth-First search by **repeated applications of Depth-First search**, each up to a preset depth limit. This is called **Progressive Deepening Search (PDS)**:

Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

Pick **first** element of Q; Add path extensions to **front** of Q.

Progressive Deepening Search

Best of Both Worlds

Depth-First Search (DFS) has small space requirements (linear in depth), but has major problems:

- DFS can run forever in search space with infinite length paths
- DFS does not guarantee finding the shallowest goal

Breadth-First Search (BFS) guarantees finding the shallowest goal states, even in the presence of infinite paths, but has one great problem:

- BFS requires a great deal of space (exponential in depth)

Progressive Deepening Search (PDS) has the advantages of both DFS and BFS.

- PDS has small space requirements (linear in depth)
- PDS guarantees finding the shallowest goal

back

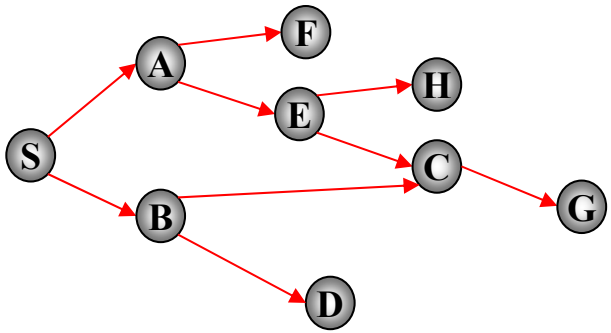
Progressive Deepening Search

Pseudocode

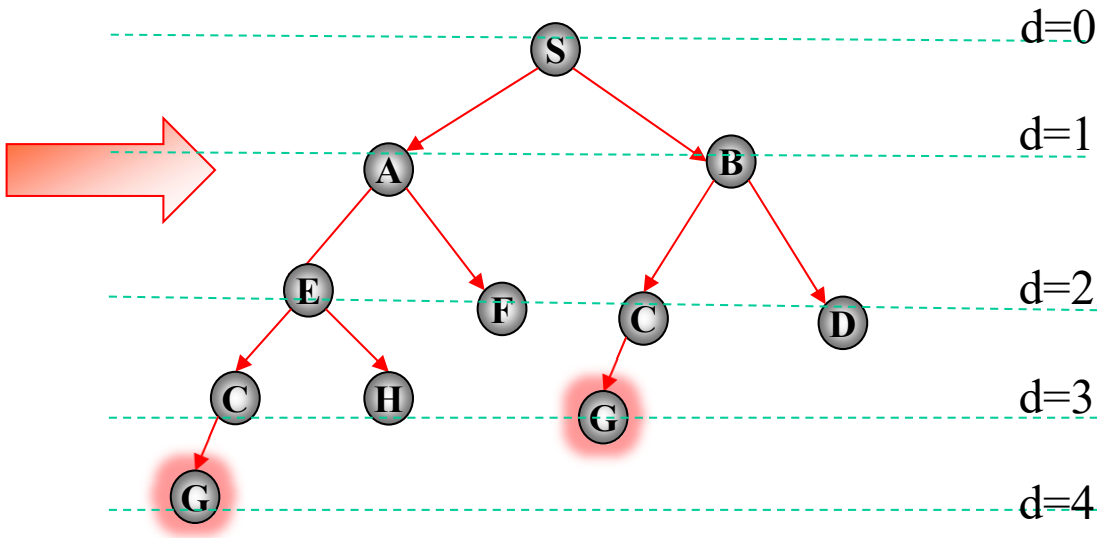
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

Example: Apply PDS on the given graph.

GRAPH



SEARCH TREE



Progressive Deepening Search
without the **Visited List**

back

SEARCH

Progressive Deepening Search

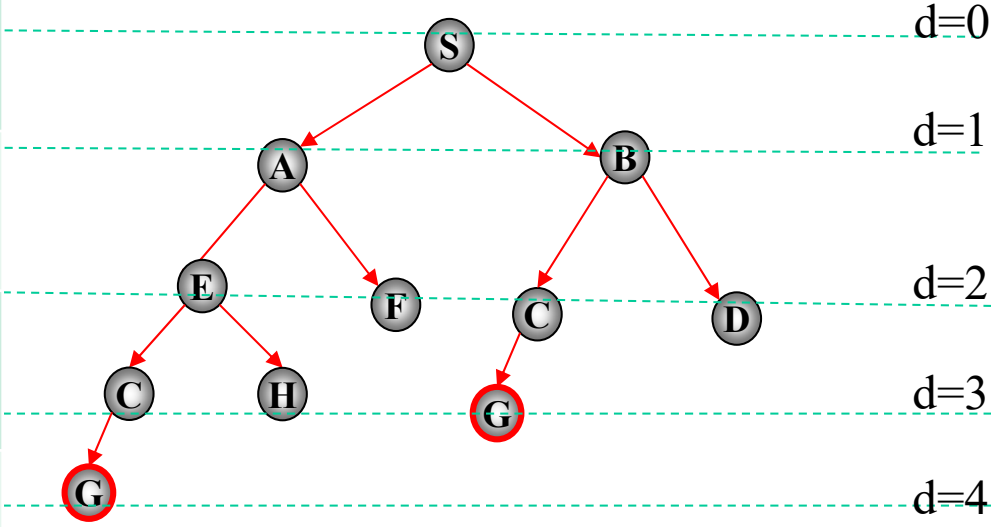
Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

C=1

Step	Dequeued	Enqueued
0		(S^0) *depth=0*

SEARCH TREE



Pick **first** element of Q; Add path extensions to **front** of Q.

back

Progressive Deepening Search

Pseudocode

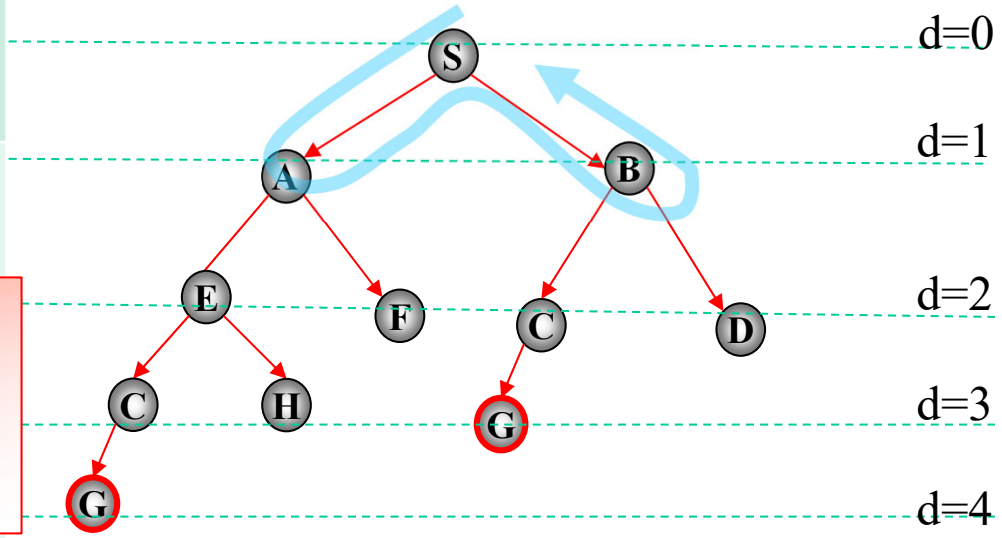
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

C=1

Step	Dequeued	Enqueued
0		(S^0) *depth=0*
1	(S^0)	(AS^1) (BS^1)

S^0 has depth level 0.
Since it is less than the setting for parameter C (i.e. C=1), that means we will allow to expand it

SEARCH TREE



Notation: AS^1 *depth=1*

Pick **first** element of Q; Add path extensions to **front** of Q.

SEARCH

Progressive Deepening Search

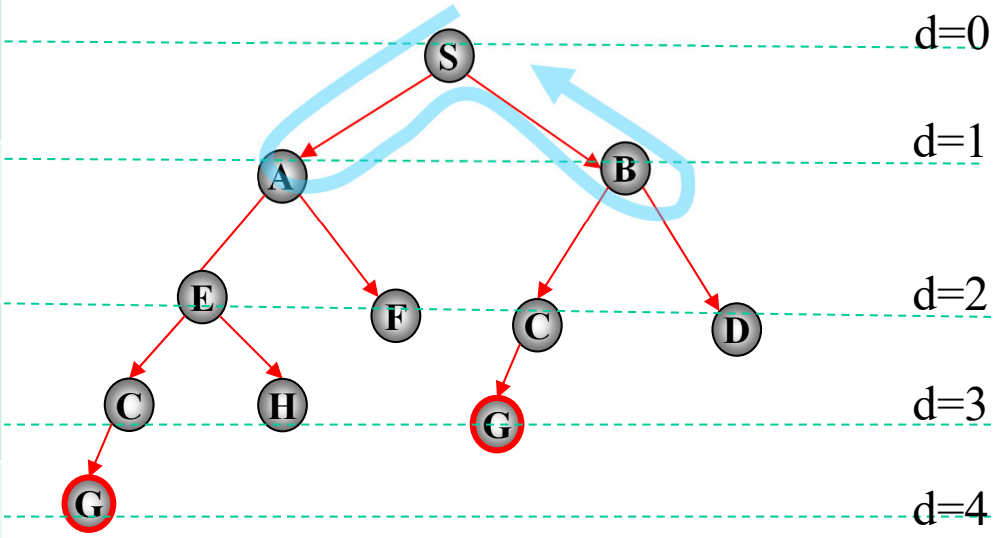
Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

C=1

Step	Dequeued	Enqueued
0		~~(S^0)~~ *depth=0*
1	(S^0)	~~(AS^1)~~ (BS^1)
2	(AS^1) . *already at depth=1*	~~(BS^1)~~
3	(BS^1) . *already at depth=1*	*Q is empty.* *No Solution.*

SEARCH TREE



Notation: AS^1 → *depth=1*

Pick **first** element of Q; Add path extensions to **front** of Q.

back

SEARCH

Progressive Deepening Search

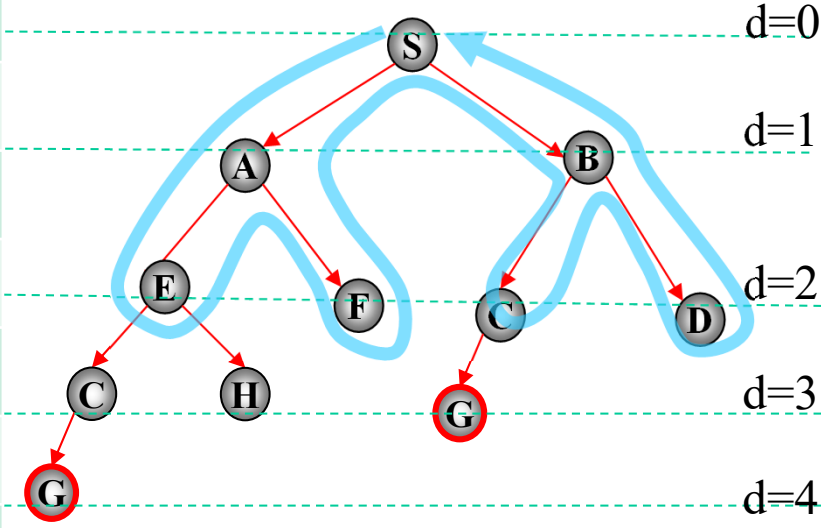
Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

➡ **C=2**

Step	Dequeued	Enqueued
0		(S^0)
1	(S^0)	(AS^1) (BS^1)
2	(AS^1)	(EAS^2) (FAS^2) (BS^1)
3	(EAS^2) .	(FAS^2) (BS^1)
4	(FAS^2) .	(BS^1)
5	(BS^1)	(CBS^2) (DBS^2)
6	(CBS^2) .	(DBS^2)
7	(DBS^2) .	*No Solution.*

SEARCH TREE



Pick **first** element of Q; Add path extensions to **front** of Q.

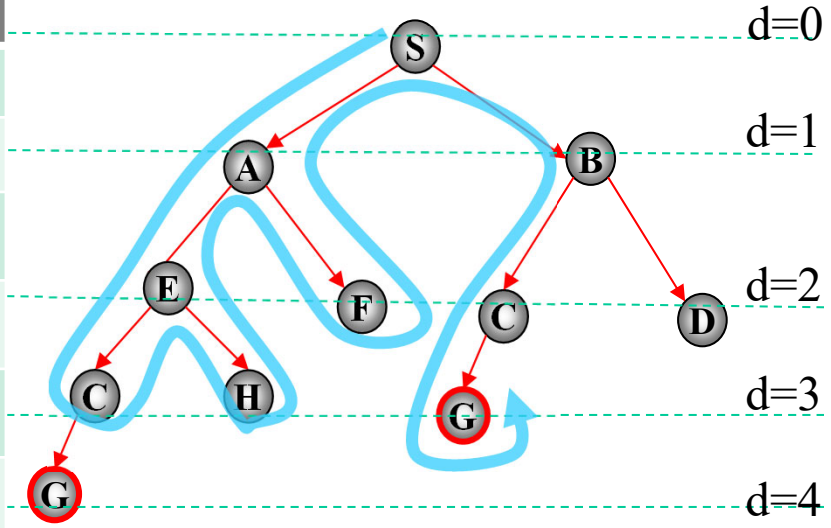
Progressive Deepening Search

C=3

Step	Dequeued	Enqueued
0		(S^0)
1	(S^0)	(AS^1) (BS^1)
2	(AS^1)	(EAS^2) (FAS^2) (BS^1)
3	(EAS^2)	($CEAS^3$)($HEAS^3$)(FAS^2) (BS^1)
4	($CEAS^3$) .	($HEAS^3$)(FAS^2) (BS^1)
5	($HEAS^3$) .	(FAS^2) (BS^1)
6	(FAS^2)	(BS^1)
7	(BS^1)	(CBS^2) (DBS^2)
8	(CBS^2)	($GCBS^3$) (DBS^2)
9	($\underline{G}CBS^3$)	(DBS^2)

Solution found!

SEARCH TREE



path = S-B-C-G

Is this the shallowest goal in the tree?

Notation: AS^1 → *depth=1*

Progressive Deepening Search

Best of Both Worlds

Isn't **Progressive Deepening Search** (PDS) too expensive?

- PDS looks at the same nodes over and over again…

Progressive Deepening Search

Best of Both Worlds

Isn't **Progressive Deepening Search** (PDS) too expensive?

- In small graphs?
- In exponential trees, time is dominated by deepest search

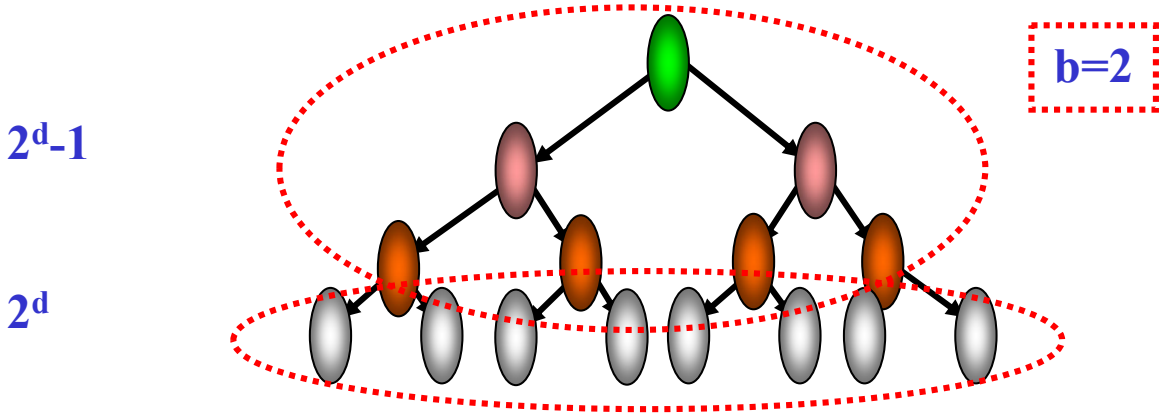
Progressive Deepening Search

Isn't **Progressive Deepening Search** (PDS) too expensive?

- In small graphs?
- In exponential trees, time is dominated by deepest search

For example, if branching factor is 2, then the number of nodes at depth d (also the **worst case running time at depth d**) is $\mathbf{2^d}$ while the total number of nodes in all previous levels is $(2^d - 1)$, so the difference between looking at the whole tree versus only the deepest level is at worst a **factor of 2** in performance.

At level **d**, compare the work done by **Breadth-First Search vs**. work done by **PDS**



Progressive Deepening Search

Ratio of the work done by PDS to that done by a single DFS

Compare the **ratio of average time** spent on **PDS** with average time spent on a **single DFS** with the full depth tree:

(Average time for PDS)/(Average time for DFS) ≈ **(b+1)/(b-1)**

b	ratio
2	3
3	2
5	1.5
25	1.08
100	1.02

Progressive Deepening Search

Ratio of the work done by PDS to that done by a single DFS

Compare the **ratio of average time** spent on **PDS** with average time spent on a **single DFS** with the full depth tree:

(Average time for PDS)/(Average time for DFS) ≈ **(b+1)/(b-1)**

Progressive Deepening is an effective strategy for difficult searches.

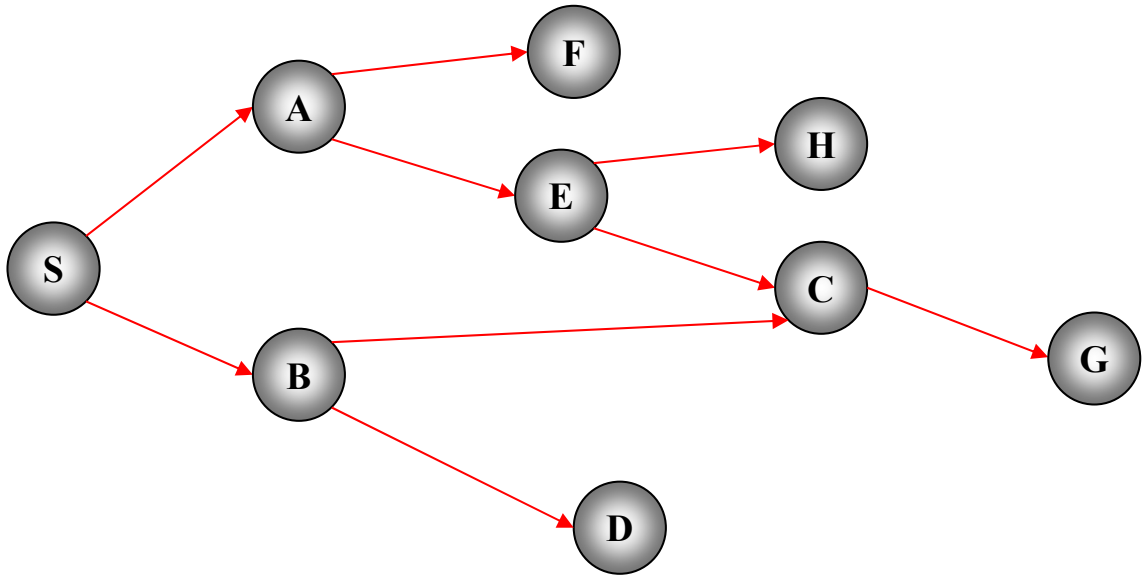
b	ratio
2	3
3	2
5	1.5
25	1.08
100	1.02

Progressive Deepening Search with the **Visited List**

back

Progressive Deepening Search

Will the use of Visited List help the algorithm find the shallowest goal?



Pseudocode

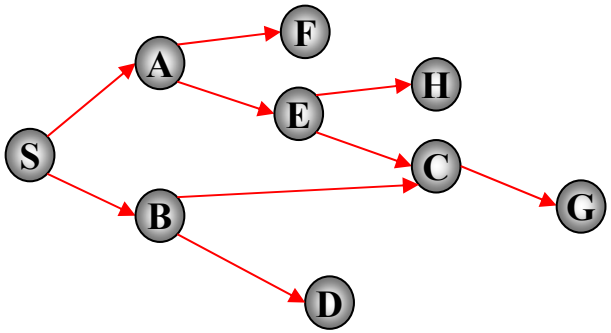
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

back

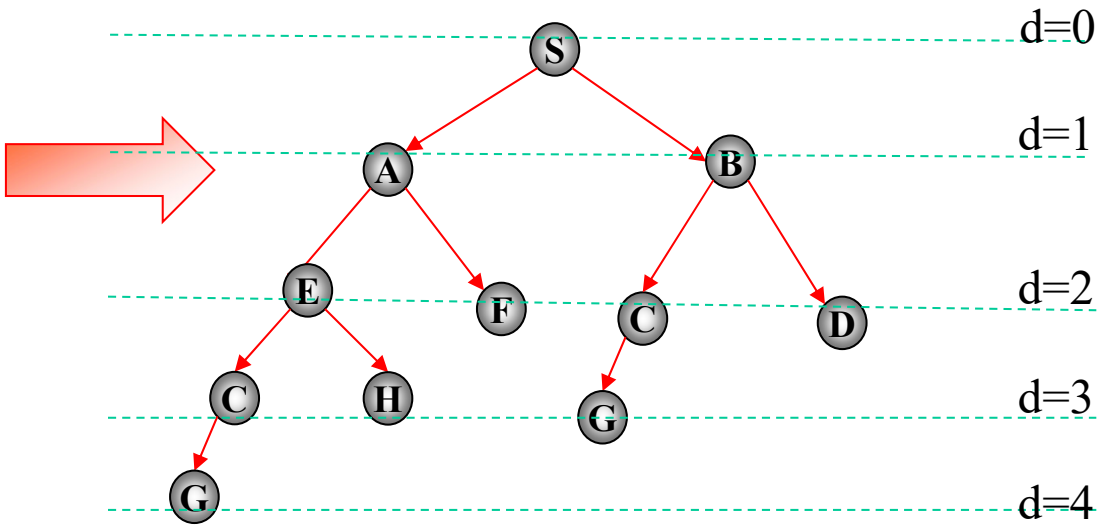
Progressive Deepening Search

Given the graph below, simulate PDS with and without using the Visited List.

GRAPH



SEARCH TREE



Pseudocode

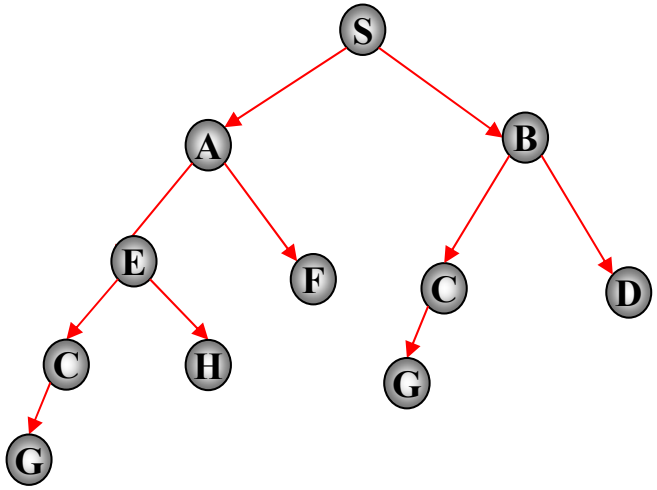
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

back

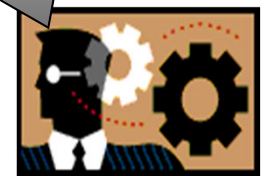
Progressive Deepening Search

Depth-Limited Depth-First Search with Visited List (max depth=3)

SEARCH TREE



Can we improve PDS by using the Visited List?



back

Progressive Deepening Search
with the **Visited List**

Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** (**with Visited List)** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

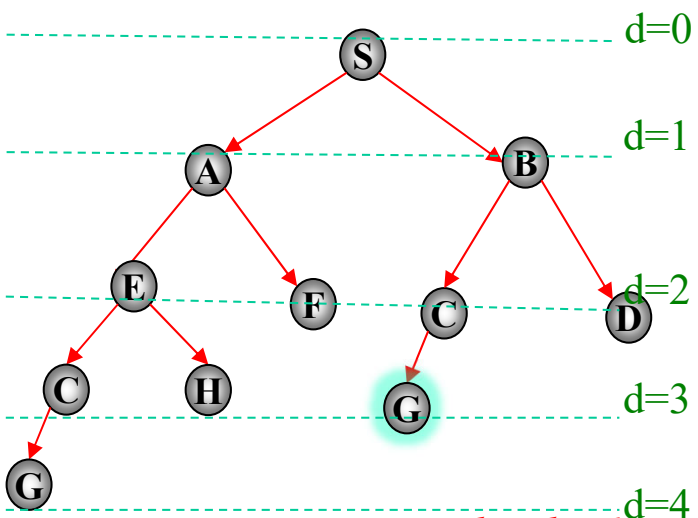
back

Progressive Deepening Search

Depth-Limited Depth-First Search with Visited List (max depth=3)

Recall previous PDS solution that does not use the Visited List

SEARCH TREE



Notation: AS^1

Step	Dequeued	Enqueued	Visited
0		(S^0)	S
1	(S^0)	$\underline{(AS^1)}(BS^1)$	S,A,B
2	(AS^1)	$\underline{(EAS^2)}(FAS^2)(BS^1)$	“ ,E,F
3	(EAS^2)	$\underline{(CEAS^3)}(HEAS^3)(FAS^2)(BS^1)$	“, C,H
4	$(CEAS^3)$. d=3	$\underline{(HEAS^3)}(FAS^2)(BS^1)$	“
5	$(HEAS^3)$. d=3	$\underline{(FAS^2)}(BS^1)$	“
6	(FAS^2)	$\underline{(BS^1)}$	“
7	(BS^1)	$\underline{(DBS^2)}$	“, D
8	(DBS^2)	Q is empty! No solution found.	

back

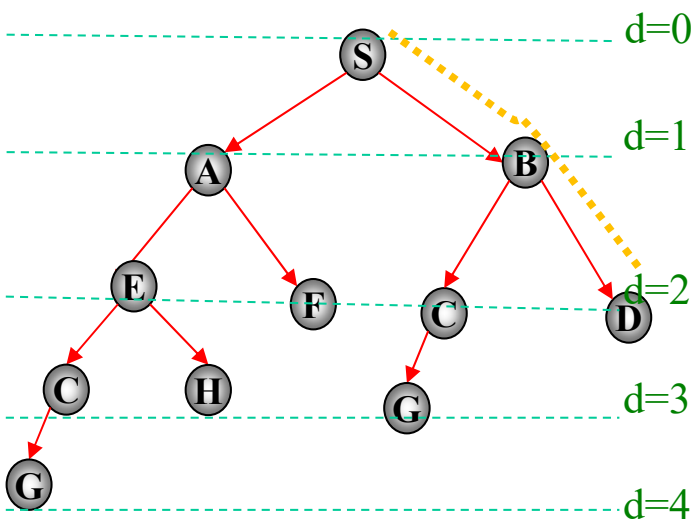
SEARCH

Progressive Deepening Search

Depth-Limited Depth-

The use of the Visited List caused the algorithm to miss finding a solution!

How can we remedy this?



Step	Dequeued	Enqueued	Visited
0		(S^0)	S
1	(S^0)	(AS^1)(BS^1)	S,A,B
2	(AS^1)	(EAS^2)(FAS^2)(BS^1)	“ ,E,F
3	(EAS^2)	($CEAS^3$)($HEAS^3$)(FAS^2)(BS^1)	“, C,H
4	($CEAS^3$). d=3	($HEAS^3$)(FAS^2)(BS^1)	“
5	($HEAS^3$). d=3	(FAS^2)(BS^1)	“
6	(FAS^2)	(BS^1)	“
7	(BS^1)	(DBS^2) Not extended to **C** as it is already in the Visited List	“, D
8	(DBS^2)	Q is empty! No solution found.	

back

Progressive Deepening Search with the Non-Strict Visited List

Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** (**with Non-Strict Visited List)** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

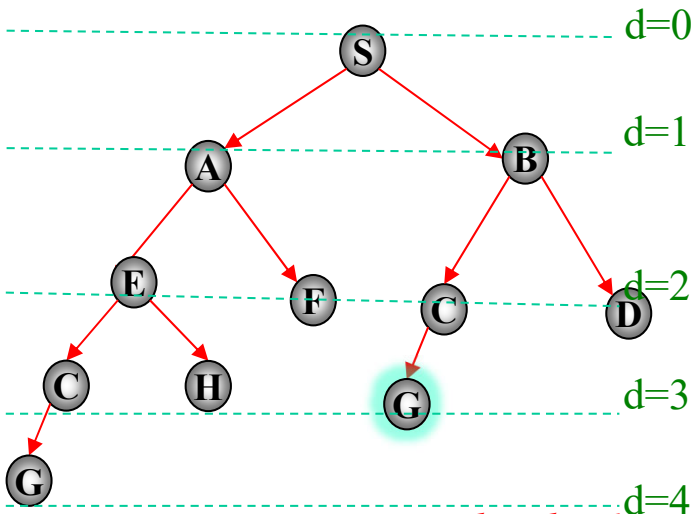
back

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0

SEARCH TREE



Notation: AS^1

We will keep track of the depth at which a state has been visited.

SEARCH

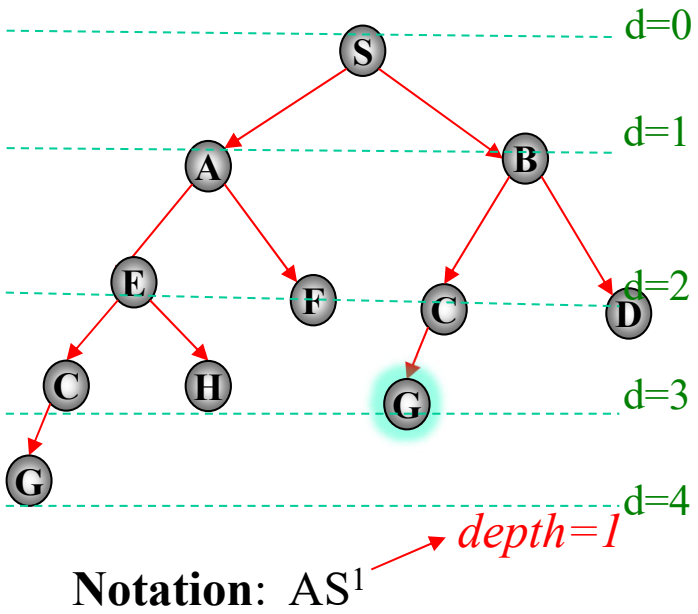
Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0
1	(S^0)	($\underline{AS^1}$)(BS^1)	S^0,A^1,B^1

We will keep track of the depth at which a state has been visited.

SEARCH TREE



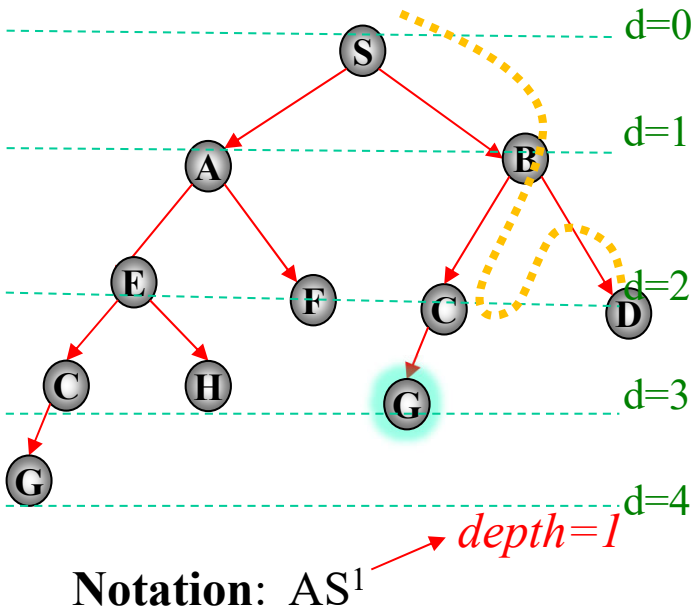
Notation: AS^1

SEARCH

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

SEARCH TREE



Notation: AS^1

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0
1	(S^0)	$(AS^1)(BS^1)$	S^0, A^1, B^1
2	(AS^1)	$(EAS^2)(FAS^2)(BS^1)$	“ $, E^2, F^2$
3	(EAS^2)	$(CEAS^3)(HEAS^3)(FAS^2)(BS^1)$	“, ~~C^3~~, H^3
4	$(CEAS^3)$. d=3	$(HEAS^3)(FAS^2)(BS^1)$	“
5	$(HEAS^3)$. d=3	$(FAS^2)(BS^1)$	“
6	(FAS^2)	(BS^1)	“
7	(BS^1)	(CBS^2) (DBS^2)	“, C^2, D^2

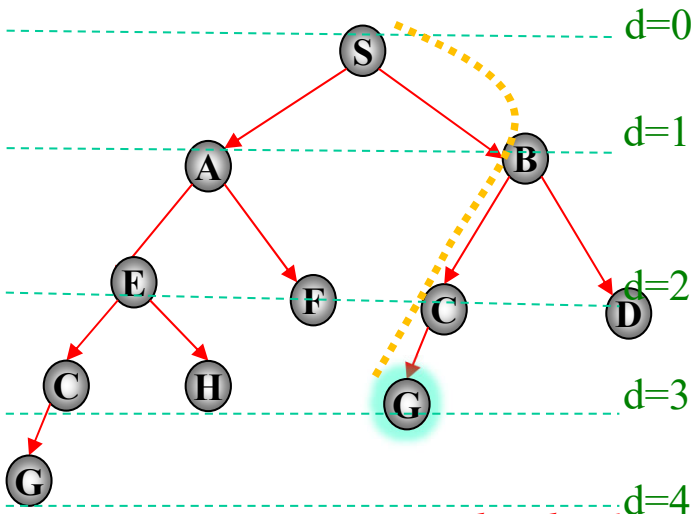
extended to C as even if it is already in the Visited List, since the path is at a lower depth-level

SEARCH

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

SEARCH TREE



Notation: AS^1

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0
1	(S^0)	$(AS^1)(BS^1)$	S^0, A^1, B^1
2	(AS^1)	$(EAS^2)(FAS^2)(BS^1)$	" $, E^2, F^2$
3	(EAS^2)	$(CEAS^3)(HEAS^3)(FAS^2)(BS^1)$	", ~~C^3~~, H^3
4	$(CEAS^3)$. d=3	$(HEAS^3)(FAS^2)(BS^1)$	"
5	$(HEAS^3)$. d=3	$(FAS^2)(BS^1)$	"
6	(FAS^2)	(BS^1)	"
7	(BS^1)	(CBS^2) (DBS^2)	", C^2, D^2
8	(CBS^2)	$(GCBS^3)$ (DBS^2)	", G^3
9	$(GCBS^3)$	(DBS^2) *Solution found!*	

back

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

The non-strict visited list allows the PDS algorithm to add a node whose state already appears in the visited list, provided that the duplicate state has a lower depth-level than the previous state added into the Visited List. This prevents the algorithm from missing the shallowest goal in the tree.